GS — CLOUD SECURITY PREVENTION

e DEVSECOPS

RELATÓRIO TÉCNICO

Bruno Kenji Sakimoto

RM567502

1. Resumo executivo

Este projeto implementa um planner de tarefas web com interface React e uma API REST para persistência em SQLite. A implantação foi preparada para concentrar front-end e API em um único Azure App Service Linux, executado por um processo Node.js. O GitHub Actions automatiza build, testes, empacotamento e publicação.

A aplicação usa Azure App Service Linux, runtime Node.js 24 e região East US 2. Inicialmente foi usado o plano Free (F1); para testar a aplicação, o responsável optou posteriormente pelo plano B1 (Basic, pago).

O deploy foi concluído, e o responsável confirmou que a aplicação funcionou corretamente. A captura final mostra a interface com o CSS carregado.

Objetivo e escopo

Demonstrar uma aplicação web com separação entre interface e API, persistência de dados, integração contínua e hospedagem como serviço gerenciado. O relatório descreve os componentes e as evidências da implantação concluída.

Repositório

C

2. Arquitetura da aplicação

O navegador acessa a aplicação por HTTPS. O processo principal Node.js inicia app-host.js, que encaminha as rotas /api para o Express, publica os arquivos gerados do front-end e envia as demais rotas ao middleware Nitro/TanStack Start. Isso concentra front-end e API em um único App Service.

Componente	Responsabilidade
Navegador	Carrega a interface e chama endpoints relativos em /api.
Azure App Service	Hospeda o processo Node.js e fornece a URL HTTPS gerenciada.
app-host.js / Express	Ponto de entrada único; monta a API, serve .output/public e encaminha as outras rotas ao front-end.
API Express	Implementa operações REST de tarefas e subtarefas sob /api.
SQLite	Armazena registros em um arquivo tarefas.db no caminho definido por DB_PATH.

Fluxo principal: navegador -> HTTPS -> App Service -> (front-end TanStack Start ou API Express em /api) -> arquivo SQLite. O front-end e a API compartilham o mesmo domínio, então o build de produção usa VITE_API_URL=/api e não depende de CORS entre serviços.

Rotas da API

A API inclui status em GET /api/, listagem em GET /api/tarefas, consulta por identificador, criação (POST), atualização (PUT) e exclusão (DELETE). As operações usam respostas HTTP e JSON; a criação requer um título.

3. Linguagens e tecnologias

Tecnologia	Uso neste projeto
TypeScript	Linguagem predominante do front-end e das rotas da aplicação.
React	Biblioteca de componentes da interface.
TanStack Start / Router	Renderização e roteamento da aplicação; build gerado com Vite/Nitro.
JavaScript / Node.js 24	Runtime e servidor de aplicação; ponto de entrada app-host.js.
Express	Servidor HTTP e roteamento da API REST.
SQLite / SQL	Banco relacional embarcado em arquivo, acessado por node:sqlite.
GitHub Actions	Automação de instalação, build, testes e publicação no Azure.

O runtime de hospedagem configurado é Node.js 24. O código da API usa JavaScript e o módulo nativo node:sqlite; não há um servidor de banco de dados separado nessa arquitetura.

4. Nuvem, plano e armazenamento

Azure App Service (PaaS)

O Azure App Service é uma plataforma como serviço (PaaS): o Azure administra a infraestrutura física e parte da camada de sistema operacional/runtime, enquanto a equipe configura o aplicativo, o runtime, variáveis de ambiente e implantação. Não é uma máquina virtual criada ou administrada pelo projeto.

O plano atual é B1, na camada Basic (paga), conforme informado pelo responsável pelo projeto. B1 é o SKU do plano do App Service, não de uma máquina virtual. O plano Free (F1) aparece apenas em capturas históricas anteriores à mudança.

Cotas observadas no portal

A Figura 1 registra as cotas do plano Free (F1) antes da mudança para B1. Portanto, os valores da captura são históricos e não descrevem o plano atual. O B1 foi escolhido para testar a aplicação com o tier Basic. Consulte o portal Azure para os recursos e limites exatos associados ao plano e à região.

Persistência de dados

SQLite grava em arquivo. O código usa DB_PATH e, sem configuração, aponta para ./tarefas.db; no App Service isso pode ficar no diretório de publicação e não deve ser tratado como armazenamento permanente. Configure DB_PATH=/home/tarefas.db e confirme que o armazenamento persistente do App Service está habilitado para /home.

Não foi identificado um Azure Storage Account ou Azure SQL Database como parte da solução atual. O arquivo SQLite é adequado para uma demonstração de instância única, mas não é um banco compartilhado entre múltiplas instâncias. Para alta disponibilidade, escala horizontal, cópias de segurança ou uso concorrente maior, recomenda-se migrar para uma base gerenciada, por exemplo Azure SQL ou Azure Database for PostgreSQL, e planejar backups.

5. Build e implantação contínua

O workflow único .github/workflows/deploy-app.yml executa o processo abaixo na branch main ou por acionamento manual:

- Obtém o código e configura Node.js 24 no runner do GitHub Actions.
- Instala dependências reproduzíveis com npm ci.
- Gera o front-end com VITE_API_URL=/api e executa npm test.
- Remove dependências de desenvolvimento, prepara o pacote de produção com app-host.js, API, node_modules e .output.
- Publica o pacote no App Service pelo azure/webapps-deploy@v3.

Configuração do repositório: a variável de Actions APP_NAME deve conter o nome curto exato do App Service; o secret APP_PUBLISH_PROFILE deve guardar o perfil de publicação XML correspondente. O arquivo XML é uma credencial: mantenha-o apenas no secret do GitHub e nunca o inclua no repositório ou neste relatório.

No App Service, o comando de inicialização esperado é npm start. O script start executa node app-host.js. Configure também DB_PATH=/home/tarefas.db e valide a persistência de /home.

Segurança e operação

- Não compartilhar publish profiles, senhas, tokens ou identificadores de assinatura em capturas públicas.
- Aplicar privilégio mínimo aos acessos do repositório e rotacionar o perfil de publicação se ele for exposto.
- Confirmar HTTPS, variáveis de ambiente e logs de inicialização após cada deploy.
- Usar plano com recursos e disponibilidade compatíveis com o objetivo antes de tratar a aplicação como serviço de produção.

6. Estado final da aplicação

A execução final do GitHub Actions foi concluída, e o responsável pelo projeto confirmou que a aplicação funcionou corretamente após o deploy. A captura final confirma que a interface carregou com o estilo visual esperado.

A captura inicial registrava o problema de CSS; após a correção e o novo deploy, a captura final mostra a interface estilizada, sem o problema visual anterior. A correção serve os arquivos de .output/public antes do middleware TanStack. Os testes locais anteriores também confirmaram CSS, JavaScript e rotas da API.

Validação final registrada

- Deploy da versão integrada realizado pelo workflow do GitHub Actions.
- Interface acessada pela URL HTTPS do App Service e apresentada com o layout estilizado.
- Arquivos CSS e JavaScript compilados servidos pelo host Express após a correção implementada.
- Os testes locais anteriores confirmaram CSS com status 200 e MIME text/css, além de respostas das rotas da API.
- O responsável pelo projeto confirmou o funcionamento correto da aplicação ao final; a captura registra a interface sem o erro visual anterior.
- O plano atual B1 foi escolhido para testar a aplicação. A captura de cotas F1 representa o estágio anterior; consulte o portal Azure para acompanhar o plano e o consumo atuais.

Conclusão: a arquitetura de instância única foi implantada e o pipeline automatiza o deploy. A captura final confirma que a interface está estilizada, e o responsável confirmou que a aplicação funcionou corretamente. O plano B1 (Basic, pago) foi escolhido para testar a aplicação. O SQLite requer armazenamento persistente e foi mantido em uma instância; essas limitações devem ser consideradas caso a solução evolua para produção.

7. Registros visuais

As imagens registram etapas da implantação: cotas do plano F1, execução bem-sucedida do workflow e estado final estilizado da aplicação, após a resolução do problema de CSS.

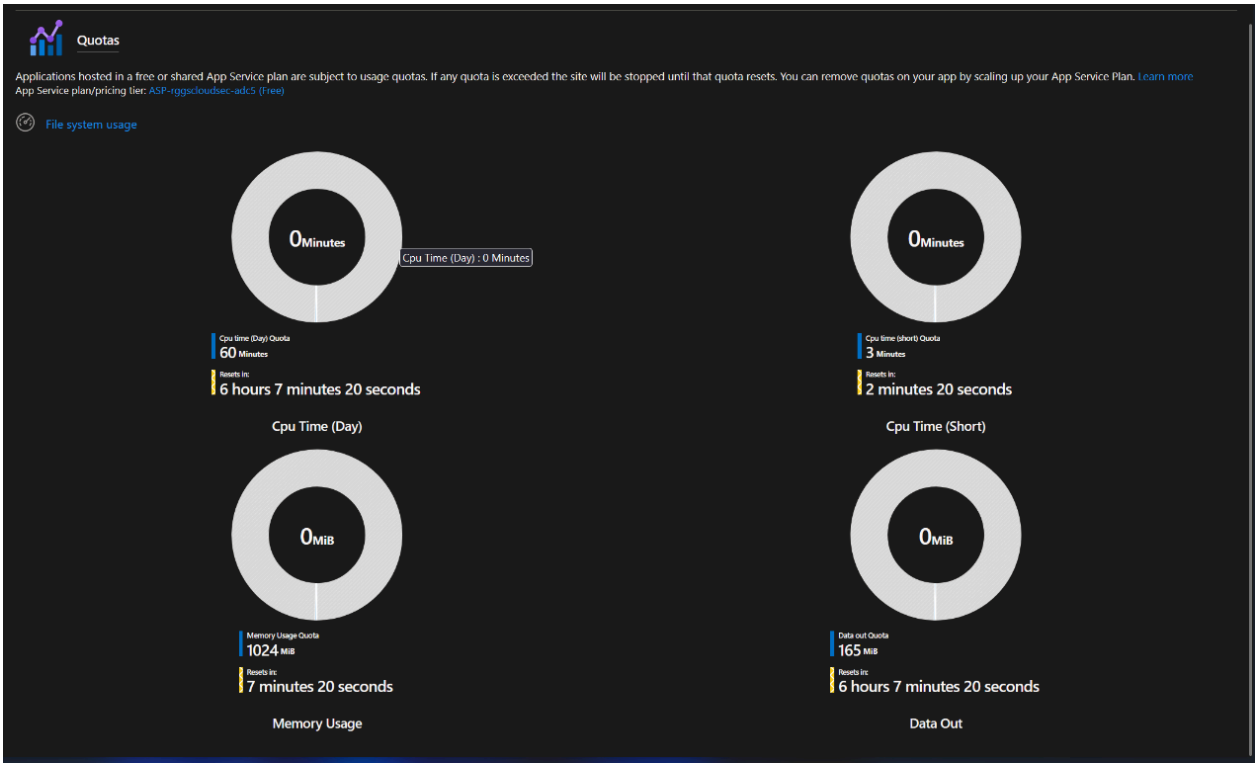


Figura 1 - Cotas do plano Free (F1) registradas antes da mudança para B1; captura histórica, não representa a configuração atual.

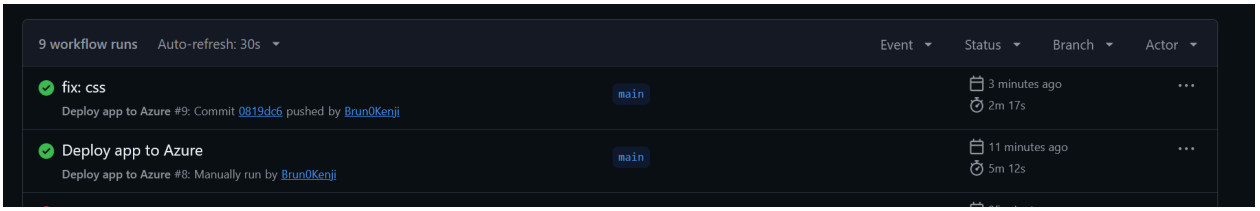


Figura 2 - Execução do workflow registrada como concluída no GitHub Actions.



Figura 3 - Estado final do planner no App Service HTTPS, com estilos carregados; o problema visual anterior foi resolvido (captura fornecida após a validação final).

8. Passo a passo: da criação do App Service ao deploy

1. Preparar o repositório e definir o destino

Confirme que o repositório contém o workflow .github/workflows/deploy-app.yml e que o código inicia com npm start. O arquivo PublishSettings analisado corresponde a wa-gs-cloud, na região East US 2, mas não confirma o sistema operacional nem o runtime configurado. Há também o workflow legado .github/workflows/main_wa-gs-cloudsec.yml, que publica em wa-gs-cloudsec. Antes de criar ou publicar, decida qual App Service será o destino. Para este procedimento, use o nome do novo aplicativo em todos os locais e desative ou ajuste o workflow legado para impedir uma publicação acidental no recurso errado.

2. Criar o App Service no Portal Azure

Entre no portal.azure.com e selecione Criar um recurso > Aplicativo Web (Web App). Na guia Básico, selecione a assinatura, crie ou escolha um grupo de recursos, informe um nome exclusivo para o aplicativo e escolha Publicar: Código. Selecione pilha de runtime Node 24, sistema operacional Linux e região East US 2 (ou a região aprovada para o projeto). Em Plano do App Service, crie um plano Linux na camada Basic B1, conforme a configuração descrita no relatório. B1 é pago: confira preço, assinatura, região e SKU antes de continuar. Na guia Implantação, não gere um segundo workflow automático se for manter o deploy-app.yml existente. Revise as opções, selecione Examinar + criar e, após a validação, Criar. Aguarde a implantação do recurso.

3. Registrar e conferir os dados do aplicativo

Abra o recurso criado e, em Visão geral, confirme o nome exato, o status e o domínio HTTPS padrão. Confirme também que o aplicativo é Linux com Node 24 e que o plano é o B1 pretendido. O domínio padrão deve corresponder ao nome escolhido; não reutilize automaticamente o perfil de publicação de outro aplicativo.

4. Configurar inicialização e banco SQLite

Em Configuração > Configurações gerais, defina o comando de inicialização como npm start, usado pelo projeto para executar node app-host.js; o servidor escuta a porta PORT fornecida pelo App Service. Em Configuração > Configurações do aplicativo, adicione DB_PATH com valor /home/tarefas.db e salve as alterações. Confirme que o armazenamento persistente do App Service está habilitado/disponível em /home e reinicie o aplicativo se o portal solicitar. O banco atual é SQLite em arquivo e atende a esta arquitetura de instância única; não configure a connection string SQL Server presente no perfil, pois o código usa DB_PATH e não essa conexão.

5. Configurar a publicação segura no GitHub

No Portal Azure, baixe o perfil de publicação do App Service recém-criado somente se for usar a autenticação já prevista no workflow. No GitHub, abra o repositório em Settings > Secrets and variables > Actions. Crie a variável APP_NAME com o nome exato do recurso e o secret APP_PUBLISH_PROFILE com o XML completo desse mesmo recurso. Não coloque o XML em arquivos rastreados pelo Git, no relatório, em capturas ou nos logs. O workflow atual usa publish profile; se a política do Azure bloquear a autenticação básica de publicação, prefira configurar OpenID Connect (OIDC) e adaptar o workflow em vez de habilitar credenciais básicas sem aprovação.

6. Evitar workflows e destinos conflitantes

Antes do deploy, confira .github/workflows/deploy-app.yml: ele lê APP_NAME e APP_PUBLISH_PROFILE e publica no recurso indicado. Confira também .github/workflows/main_wa-gs-cloudsec.yml: ele contém outro nome de aplicativo e outro método

de autenticação. Desative, remova ou atualize o workflow legado conforme a decisão do projeto, para que apenas o pipeline aprovado publique e que o nome do App Service e as credenciais sejam sempre do mesmo recurso.

7. Executar o primeiro deploy

No GitHub, abra Actions e selecione Deploy app to Azure. Para iniciar manualmente, use Run workflow na branch main; alternativamente, envie para main uma alteração abrangida pelos caminhos configurados no workflow. Acompanhe a execução até a conclusão. O pipeline instala as dependências com npm ci, compila o front-end com VITE_API_URL=/api, executa os testes, prepara os arquivos de produção e envia o pacote ao App Service. Resolva eventuais falhas antes de repetir a publicação.

8. Validar o site, a API e a persistência

Acesse o domínio HTTPS do App Service e confirme que a interface e os estilos carregam. Verifique os endpoints GET /api/ e GET /api/tarefas e teste criar, atualizar e excluir uma tarefa pela interface. Consulte os logs de inicialização e o resultado do GitHub Actions. Reinicie o aplicativo e confira se as tarefas continuam disponíveis; esse teste confirma que DB_PATH aponta para o local persistente. Se a aplicação não iniciar, confira o runtime, npm start, as configurações do aplicativo e os logs.

9. Proteger credenciais e acompanhar custos

O arquivo PublishSettings contém credenciais de publicação e pode conter dados de conexão. Mantenha-o fora do Git e restrinja o acesso ao secret APP_PUBLISH_PROFILE. Como uma cópia do perfil foi colocada no workspace, remova-a do diretório do projeto após configurar o secret; se as credenciais foram compartilhadas ou expostas fora de um ambiente confiável, revogue ou rotacione o perfil no Azure. Monitore o custo do App Service B1 e o consumo da assinatura. Para produção, reavalie as limitações do SQLite local antes de adicionar instâncias ou depender de alta disponibilidade.

Referências oficiais

Microsoft Learn — Criar um aplicativo Web Node.js no Azure App Service:
https://learn.microsoft.com/azure/app-service/quickstart-nodejs

Microsoft Learn — Implantar no Azure App Service com GitHub Actions:
https://learn.microsoft.com/azure/app-service/deploy-github-actions